

从“写得快”到“交得出”：10 倍效率背后的工程方法

如何把一个真实的软件做出来、交付出去

主讲人：邓军辉

时间：2026年7月6日

蒸汽机车不只是一辆更快的马车

蒸汽机车：一辆用煤不用草，跑的更快的“铁马”。

蒸汽火车改变运输，靠的不只是更强的动力

- 真正起作用的，是围绕它长出来的一整套系统
- 轨道、时刻表、信号、岗位分工、协同规则...

任何一次技术变革，真正产生价值的方式，从来不单是“用新工具替代旧工具”，而是“围绕新工具长出一整套新方法并有效实施”



今天的 AI 也是一样



AI 让写代码变得前所未有的快，一个页面、一个接口、一段完整逻辑，很短时间就能生成...

- 但速度不等于效率，生成不等于生产
- 需求、产品、架构、编码、交付、运维...

生成代码越来越容易时，软件工程该怎么做？

怎样把 AI 的生成能力，变成团队稳定、可靠、可持续的交付能力？

AI 时代，如何做软件工程；

会用 AI 写代码"的人比比皆是，
但"能用 AI 把软件交付出去"的人，依然稀缺；

当 AI 把"写代码"这件事变得极其廉价时，
软件工程还剩下什么？我们怎么教？学员怎么练？

**《软件工程规范》《产品设计规范》
《架构设计规范》《GitHub 过程管理规范》**

第一，过程清晰

用 GitHub 承载过程：Milestone / Issue / PR / Release / 标签，进度和历史一目了然。

第二，交付有保底

每个 Milestone 都交一个可打 tag 的版本，每 2 周一兑现。

第三，风险早暴露

每 2 周验收 + 日报 / PR 有 Review + 导师和助教及时介入、纠偏。

敏捷模型，四轮完整迭代



- 两个月，8 周，4 个 Milestone
- 每轮都是迭代：产品，架构，开发
- 每轮都要有可展示、可打 tag 的产出

MS1：战略决策与首个可串联版本



第一，产品决策

确定做什么、不做什么，形成产品Proposal

第二，搭出架构主干

“空骨架”，模块如何拆、接口怎么定、数据模型长什么样，全部用代码定义清楚，入库；

接口是真的，实现可以留空或者桩实现，但模块之间的调用关系要写出来，整个系统要能编译通过、串联成立。

交付：产品取舍决策，系统架构，Live Demo；

MS2: MVP 深化与架构落实。



- 不再只是空壳，填充真实功能
- 核心流程可演示
- 模块边界、数据模型、接口风险更清楚

验收重点：核心流程真的跑通了吗？（不需要全部功能实现）

MVP 范围内的功能全部做完，做扎实；交付一个真实用户可试用的产品；

- 功能完成
- 测试覆盖
- 可交付试用

MS4：打磨、交付与发布



1. 原则上功能冻结
2. 改 Bug、修体验、补用户文档、准备演示、部署
3. Tag V1.0

验收重点：交付真实吗？表达清楚吗？材料完整吗？

每个 Milestone 都有一个问題



MS1：方向和主干成立吗？

MS2：核心路径跑通了吗？

MS3：MVP 真的做完了吗？

MS4：交付真实、表达清楚吗？

每轮内部顺序：先产品，再架构，再开发



产品：决定要什么、不要什么

架构：决定系统如何支撑

开发：基于清楚的 Issue 和 PR 推进

产品先行、架构次之、开发最后。

产品设计是取舍，不是功能清单



“我们要做A、做B、做C”

而是要回答：

1. 针对这个问题，市面上有哪些做法？
2. 我们选了哪一种？
3. 为什么选它？
4. 放弃别的方案的理由是什么？
5. 本次明确不做什么？
6. ...？

只罗列功能、不**呈现取舍**的文档，不**算产品设计**，只能算愿望清单。

Proposal 要回答五个问题



1. 用户是谁?
2. 问题是什么?
3. 为什么值得做?
- 4. 怎么做?**
5. 怎么验收?

"怎么做"的强制顺序——先文字，后原型

先文字，后原型

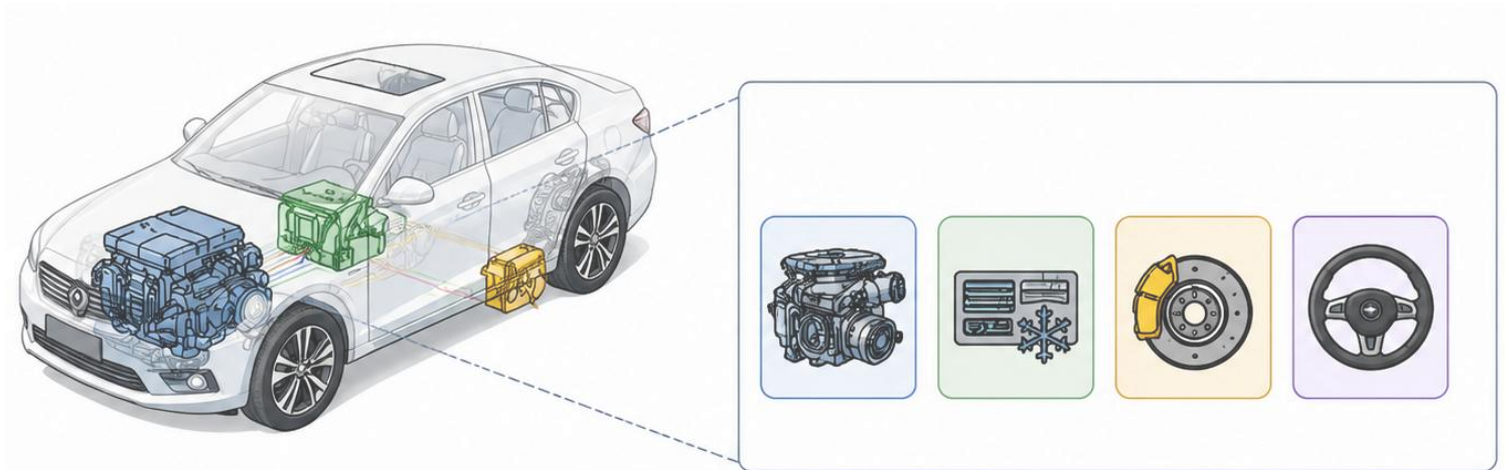


1. 先把决策用文字写清楚：解决什么问题、要什么、不要什么、边界在哪。
2. 文字定稿之后，再画原型或做 live demo。

有界面的产品两步都要做，无界面的产品（比如编译器、SDK、命令行工具）只做第一步。

真正难的是系统架构

- 模块架构：单个模块怎么实现
- 系统架构：模块怎么拆、怎么配合、接口怎么定



代码即架构：MS1 要把空骨架入库



MS1
目标



真实的代码定义
模块 / 接口 / 数据模型



方法体可空或桩实现
接口和调用关系真实



代码可编译
模块已串联



经 PR Review
后合并入库

项目仓库结构 (示例)



模块接口定义 (真实、可编译)

internal/user/user.go

```
package user

type Service interface {
    Get(ctx context.Context, id string) (*User, error)
    Create(ctx context.Context, u *User) error
}
```

internal/order/order.go

```
package order

type Service interface {
    Create(ctx context.Context, req *CreateOrderReq) (*Order, error)
}

type Order struct {
    ID string
    UserID string
    Amount int64
}
```

internal/payment/payment.go

```
package payment

type Service interface {
    Pay(ctx context.Context, orderID string, amount int64) error
}
```

internal/repo/store.go

```
package repo

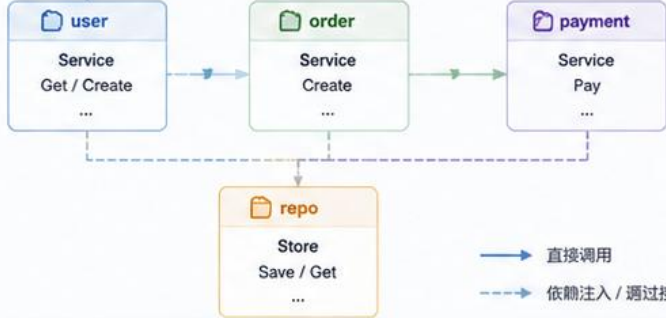
type Store interface {
    Save(ctx context.Context, key string, value []byte) error
    Get(ctx context.Context, key string) ([]byte, error)
}
```

main.go

```
package main

func main() {
    userSvc := user.NewService(repo.NewStore())
    orderSvc := order.NewService(userSvc, repo.NewStore())
    paySvc := payment.NewService(repo.NewStore())

    // 业务流调用 (真实的接口调用)
    u, _ := userSvc.Get(ctx, "u1001")
    o, _ := orderSvc.Create(ctx, &order.CreateOrderReq{
        UserID: u.ID,
        Amount: 19900,
    })
    _ = paySvc.Pay(ctx, o.ID, o.Amount)
}
```



这样做的好处



基于同一架构
接口真实存在，代码即文档



并行开发
模块解耦，互不阻塞
可持续演进
修改接口即暴露影响，安全可控



开发 A 模块
实现 user.Service



开发 B 模块
实现 order.Service



开发 C 模块
实现 payment.Service



PR Review
代码可编译，联调通过



合并入库
骨架稳定，开始并行开发

架构红线： 不要用 AI 粗糙实现冒充架构



- AI 可以帮你代码实现
- 但系统架构必须由人主导
 - MS1 骨架入库务必人来书写
- 模块边界、协作关系、接口契约要讲得清

每轮复盘都看架构是否支撑交付



两个坏例子：

- 频繁大改：主干 MS1 没有定准
- 架构稳定但功能交不出来：先前的架构不支撑迭代

理想状态：主干稳定，功能持续叠加，仅微调

GitHub 过程链路图



所有动作在 GitHub 上完成，链路：Milestone → Issue → PR → Review → Merge → Release



管目标和节奏 每两周一个，挂上本轮 所有 Issue，进度一眼可见	过程的基本单元 承载工程文档草稿、任务拆解、 设计澄清。背景清楚、目标明确、 验收标准清晰。禁止空泛标题	关联对应 Issue 承载代码和 Review。 每笔改动可追溯到需求。 PR 小而频繁，及时合并	共同把关 助教和导师把关，学员互评。 关注方向对不对、改动能否 讲清楚、测试够不够	合并入库 经 Review 通过后合并到主分支， 主干保持可用，持续集成	版本交付 每个 Milestone 结束发一个 Release，写清本轮交付内容， 打上 tag
--	--	---	---	---	--



- 站会：自行组织，导师群同步，今日完成、明日计划、卡点...
- 周会：阶段性复盘、导师参与会给方向性反馈
- 日报：人物 + 事件 + 链接。

- AI 可参与每个环节，但不替代工程流程本身
- 方案理由要讲清
- 代码逻辑要理解
- 核心 PR 附关键 Prompt 和人工 Review 说明

最终交付责任在人，不在 AI。

避免：

1. 跳过 Proposal 直接写代码
2. 绕开 Issue / PR，不经过Review直接落地
3. PR 长期堆积，等到几千行一起提
4. AI 生成代码合入后，没人能讲清逻辑
5. 范围膨胀
6. 架构变成目录和技术栈清单
7. 站会不留记录，日报不带链接

最少记住这五条



1. 先设计，再开发
2. 每个 Milestone 都是一轮完整迭代
3. MS1 先把主干串起来
4. 一切经 Issue -> PR -> Review
5. 决策写下来，变更新增，不覆盖旧判断

“

两个月后，你
应该带走什么

”

一次真实、可追溯、经得起
拷问的完整交付经历

把这两个月当成一次真
正的交付即时沟通，导
师和助教会全过程陪你
们把关